高级语言程序设计期末大作业报告

报告名称：虚拟发动机性能监控系统开发

专业：数据科学与大数据技术（精英班）

班级：工科嘉定 3 班

学号：2454259

姓名：费凡

完成日期：2025.12.26

一、项目简介

虚拟发动机模拟测试界面脱胎于发动机指示和机组警告系统（EICAS），EICAS 则是用于飞机的发动机指示和告警显示画面。本次作业基于 C++代码以及 SFML3.0.2 库进行实现，基本目的是完成发动机转速动态变化的界面显示，按照 5ms 固定间隔生成四个部分模拟数据，分别为：转速传感器（检测发动机转速），EGT 传感器（检测排气温度 T），燃油余量传感器（检测燃油余量 C），燃油流速传感器（检测燃油流速 V）。通过模拟传感器的输入，进行后续的数据处理，并在界面上按照一定规则进行运行状态呈现。该项目最终可以实现发动机从启动-稳定-停车的完整运行流程，集成了大量的故障检测算法，可以实时显示运行状态及告警提示（支持正常，警戒，警告的三级预警模式），同时运行的数据与警示信息都保存到相应文件中供后续查看与分析。

二、设计思路

1. 设计思路与顶层架构

本程序采取面向对象的设计方法，并采用分层架构进行实现。

第一层：数据模拟层

1. EngineModel 类：发动机物理模型，基于状态机（STOPPED→STARTING→STABLE→STOPPING）进行实现。

2. FuelSystem 类：燃油系统模型，可模拟单油箱多发动机供油逻辑。

3. 核心算法：线性增长（启动初期）→对数增长（启动后期）→随机波动（稳定）→指数衰减（停车）

第二层：故障检测层

1. FaultDetector 类：进行故障检测，可以实现多级检测流程

2. TestScenarioGenerator 类：产生测试场景，生成 14 种预设故障

3. 检测策略：传感器故障优先 → 燃油系统异常 → 转速异常 → 温度异常

第三层：控制协调层

1. AppController 类：系统主控制器，负责协调各模块

2. 实现方式：采用固定时间步长算法，确保数据更新与界面渲染分离

第四层：可视化 UI 层

1. Gauge 类：扇形仪表组件，支持颜色编码，通过不同颜色展示运行状况

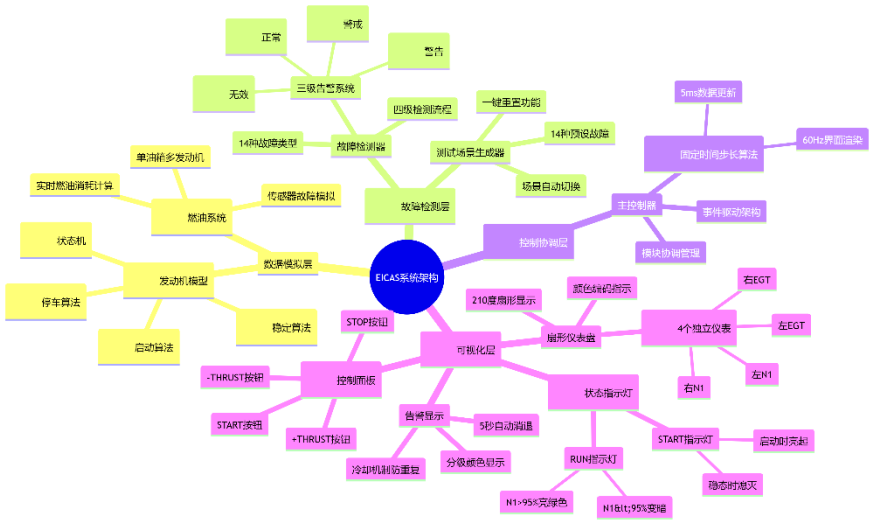
2. IndicatorPanel 类：状态指示灯（START/RUN）实现

3. ControlPanel 类：交互式控制面板，便于用户进行不同鼠标点击操作

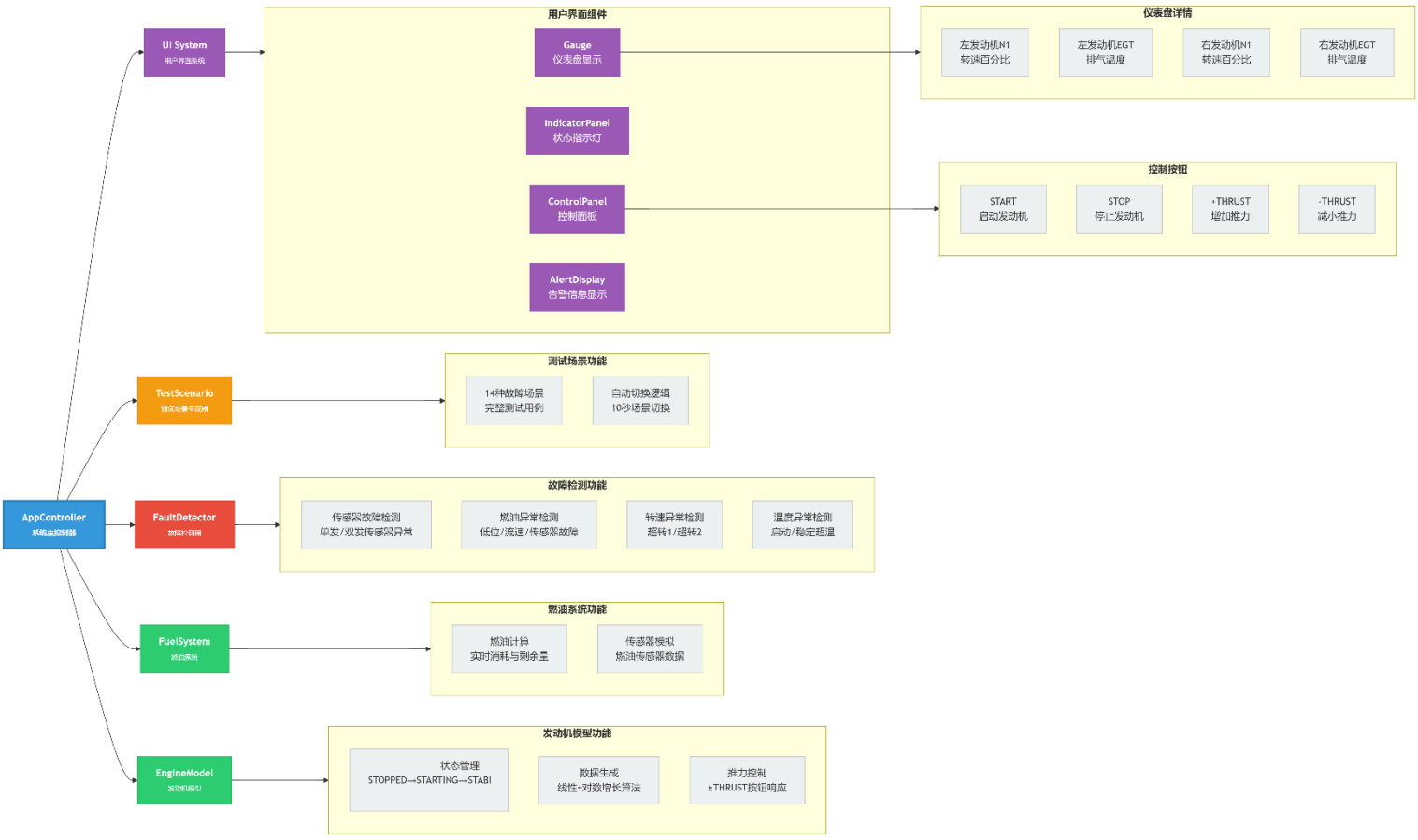
4. 告警显示系统：分级用不同颜色显示告警信息，显示 5s 后自动消退

2. 设计图示

（1）系统架构

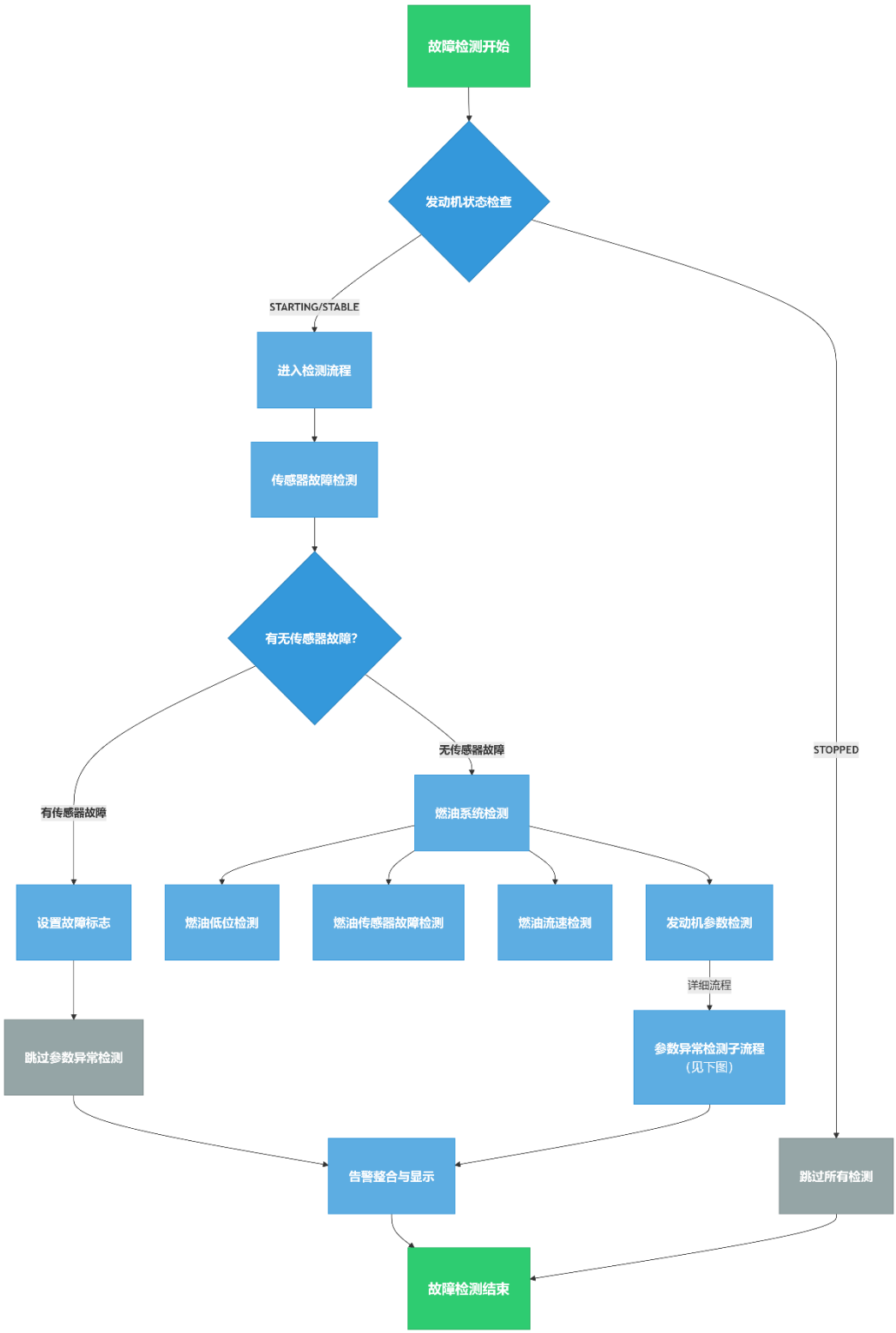


（2）模块依赖

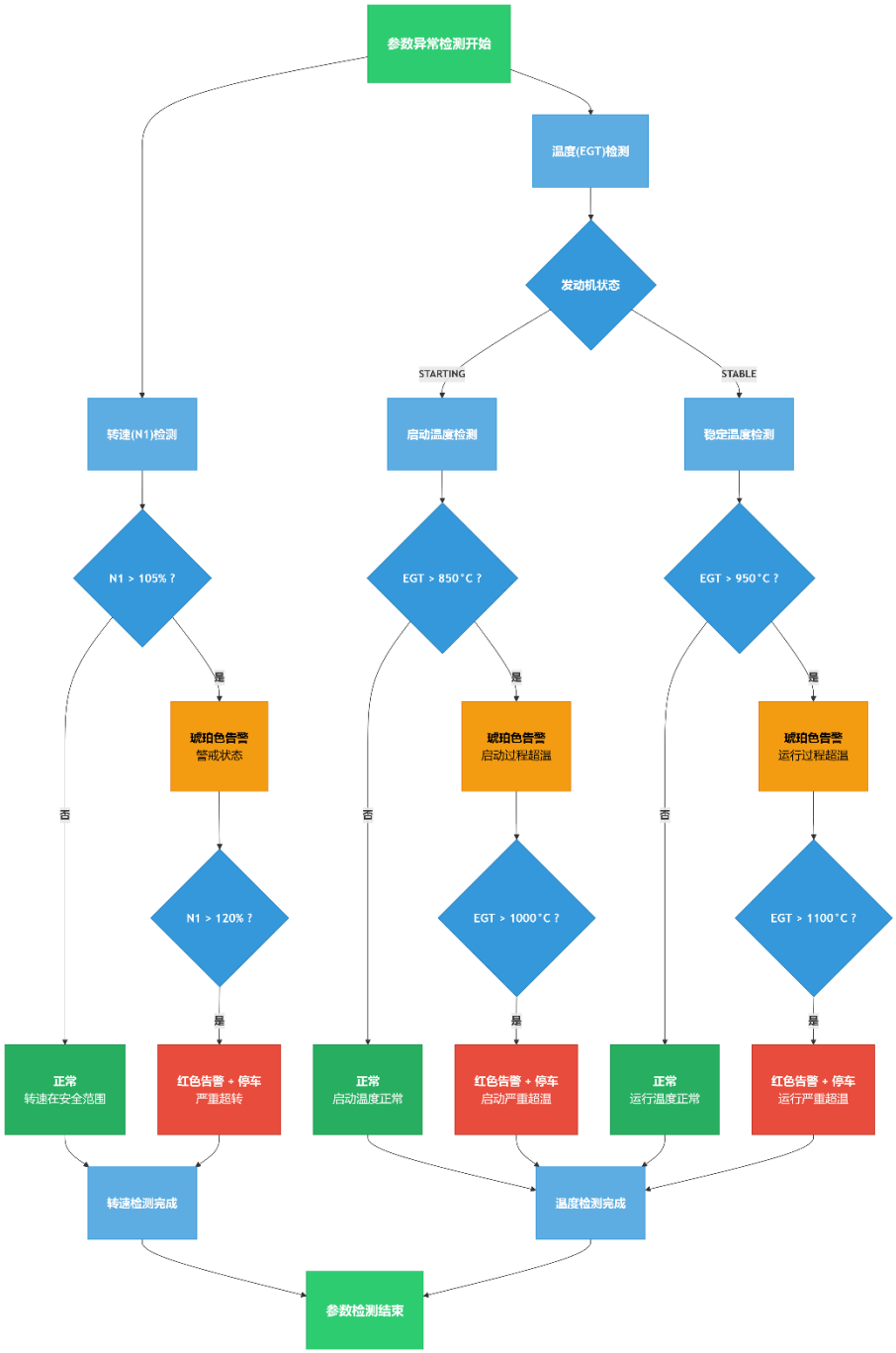


（3）故障检测流程

①主检测流程



②参数异常检测子流程



三、主要功能实现

1. 发动机状态机实现

① 用户点击 start 按钮后实现发动机启动，发动机转速分为线性增长期（前 2s）和对数增长期，直至达到稳定状态（参数保持 3%左右的动态平衡波动，模拟实际情况）。

② 实现了推力调节机制，用户可以通过"+THRUST"和"-THRUST"按钮可实时调节推力

（每增加一次推力，燃油流速增加 1 单位/秒，转速和温度在 3%-5%范围内随机增加

，减小推力同理）

③ 用户点击 STOP 按钮后，进入停车阶段。燃油流速直接归零，模拟燃油切断过程。

转速按指数衰减模型下降，确保 10 秒内完全停止。温度也从当前值向室温（20℃）指数衰减。

④ 启动过程可随时被 STOP 按钮中断，立即进入停车阶段。STOP 按钮具有最高优先级，覆盖所有其他操作指令。

2. 故障检测系统实现

① 十四种告警情况（见前面示意图展示）

② 系统采用先进的告警管理策略，避免告警泛滥和信息过载：传感器故障检测优先级最高，检测到传感器故障后，跳过后续参数异常检测；红色警告告警（WARNING）覆盖琥珀色警戒告警（CAUTION）；发动机停车状态下，跳过所有启动和运行相关的故障检测。

③ 每种故障类型设置 5 秒冷却时间，冷却期内相同故障不重复记录；告警信息在界面上持续显示 5 秒后自动消退，避免界面混乱。

④ 告警进行了正常，警戒，警告，无效几种状态的不同颜色分级处理。

3. 可视化系统实现

① 扇形仪表盘（左右各一个对称布局）分别记录 N1 发动机转速以及 EGT 发动机排气温度，角度范围控制在 0°~210°之间，按等比例进行百分比映射，角度转到一定临界值时颜色会产生变化。（每 5ms 进行一次动态刷新）若传感器故障时显示"--"符号，指针停留在 0 位。

② 状态指示灯分为 start 和 run，启动时是 start 亮起，稳态时熄灭，此时 run 亮起，稳定运行时 N1 低于 95%时会变暗，回升后又会恢复（实际运行时由于随机波动可能会出现闪烁的现象），停车后 run 熄灭。

③ 控制面板与用户的交互包括 START 按钮，STOP（优先级最高），+THRUST 按钮（增加推力）以及-THRUST 按钮。同时还有展示 14 种告警情况的 TEST SCENARIO 按钮选项，用户点击一次可以切换到下一种情况，并实现 14 种情况循环方便查看。（用户不点击时每隔 10s 自动切换到下一种场景），之后用户点击 RESET 按钮可以回到初始状态。

4. 数据管理与记录功能实现

① 正常运行数据每 5ms 记录一次，保存到相应文件名的日志文件中，表中记录了运行时间，左右发动机参数（包括转速，N1 百分比，温度等）以及燃油系统参数（包括燃油余量，总燃油流速等，同时会实时显示在运行界面上）。

② 14 种告警信息单独记录到告警日志文件，便于事后分析，同时 5 秒内同类型告警在日志中只记录一次，减少冗余信息，显示 5s 后告警提示将会消失。表中记录了实时系统时间，故障类型编码（1-14），故障级别（NORMAL、CAUTION、WARNING）以及相应的故障描述文本。

四、开发过程中遇到的问题及解决方法

1. SFML 3.0 版本适配与 API 变更挑战（核心困难）

在项目开发中，我选择了当时最新的 SFML 3.0 作为图形库，但这带来了显著的技术挑战。主要困难有 API 的大幅变更（SFML 3.0 相对于广泛使用的 2.5 版本进行了重大重构，大量类和函数的接口完全改变）；构造函数语义变化（许多类的构造方式从直接构造改为工厂模式，需要重新学习使用方式）；事件 Event 系统重构（事件处理机制从传统的枚举类型转向类型安全的 variant 模式，需要完全重写事件处理逻辑，同时需要使用 std::visit 或 event.is<T>()、event.getIf<T>()等新方法）。同时文本渲染和字体加载接口也有重大调整

解决方法：

深入理解和研读 SFML3.0 官网说明文档，理解其设计和改进理念，学习其新式构造方法，相关函数模版及语法并将其应用到代码编写中；并构建适配层和相应工具函数，分步进行相应模块化改造，最终完成整个代码文件的调配工作。

2. 控制面板按钮交互逻辑冲突

在添加推力控制按钮以及告警测试按钮后，START 和 RUN 按钮出现了严重的交互逻辑问题，指示灯状态不同步，且运行过程混乱。START 按钮和 STOP 按钮在某些状态下点击无响应等等问题。

解决方法：

重新整合状态层架构，将应用状态，UI 状态，引擎状态彻底分离，并确保几种状态同步变化，定义清晰的转化规则；确认清晰的按钮处理优先级，根据当前运行逻辑自动启用或禁用部分按钮，放宽或者严格规定某些按钮的可用条件。

3. 复杂故障检测的优先级冲突

优先级冲突不仅影响告警准确性，还可能导致严重的安全隐患，重要告警如果被次要告警抑制，可能会掩盖严重故障；而多个相关告警同时触发，造成信息过载，也会影响即时的事件处理；与此同时，逻辑上矛盾的告警若同时出现，也会影响判断。

解决方法：

设定一个规则优先级的管理机制，在首先保证系统安全的前提下，兼顾系统功能与用户体验，保证只实时显示最严重的告警信息，同时进行颜色区分以示严重性；之后进

一步优化故障检测的频率，越严重的故障检测频率越高。

五、智慧编程工具的作用

1. 相关 AI 工具在辅助我进行 SFML 新版本相关内容查找与理解中发挥了重要作用，并帮助我解决了大量语法不兼容的问题。

2. 某些复杂的算法可能很难实现，这时通过 AI 的算法提示可以给我编写代码提供一定参考，尤其是一些普通算法的优化过程的提示，对程序运行的稳定性与兼容性有很大帮助。

3. AI 还在防止内存泄漏，提高算法性能，增强代码安全性等不易注意的方面给予我必要的提示，让我受益匪浅。

六、心得体会

1. 通过本次虚拟发动机性能监控系统（EICAS）的开发，我对大型工程系统的架构设计有了颠覆性的理解。与之前做过的小型游戏与项目相比，这次项目让我深刻体会到分层的面向对象架构在复杂系统构建中的核心价值。这次的作业不再是一个简单的“功能堆砌”项目，而是综合了数据模拟、故障检测、界面显示、用户交互等多方面的大型任务。其中每个部分既相互关联又相对独立。结合题目要求，我设计了数据模拟层，故障检测层，控制协调层，可视化层等四层架构。这种分层设计的最大好处是，各个层之间的接口清晰明确，修改一处不会“牵一发而动全身”。比如当需要调整发动机的物理模型算法时，我只需修改 EngineModel 类；当要优化界面显示效果时，集中在 UI 子系统中工作即可。

2. 通过此次复杂的项目设计，我深刻理解了在设计每个类的公共接口时，要需要反复思考这个类应该对外暴露什么，隐藏什么”。例如，EngineModel 类对外部只提供了 getRPM()、getTemperature()等方法，而内部的复杂状态转换逻辑需要完全封装。这种设计不仅保护了内部状态的一致性，还使得单元测试更加容易进行。当我为故障检测逻辑编写测试用例时，可以轻松模拟各种传感器数据，而不必关心发动机内部的复杂计算过程。

3. 在告警系统的处理方面我也有了很大收获。最初，我用一长串的 if-else 语句实现故障检测，很快代码变得难以维护。当我添加很多故障之后，已经无法理清各种条件的优先级关系。重构后的解决方案是建立一个基于规则的故障检测引擎。我为每种故障定义了触发条件、严重级别、抑制关系，形成了一个清晰的检测流程图。这个过程让我认识到，复杂逻辑的处理不能仅靠直觉，需要有系统的分析方法与逻辑。同时，故障告警提示的设计让我思考良多。一方面，用户需要及时了解系统状态；另一方面，过多的告警会造成“告警疲劳”，导致重要信息被忽略。因此我结合题目要求设计提示词弹出后 5s 自动消退以避免界面混乱，设计冷却机制防止重复告警，同时利用颜色编码快速识别严重程度。这样才能设计出更加用户友好的系统。

4. 选择 SFML 3.0 作为图形库让我吃了很多苦头，因为开始并不了解升级后很多地方都

进行了重构，同时 API 也进行了变更，导致很长一段时间我都在翻阅官方文档查询相关类的构造函数，字体，场景，事件的相关设计的变化。最困难的便是事件系统，从传统的枚举到类型安全的 variant，需要完全改变思维方式。但是，当我终于理解了相关用法，能够很好处理各种事件类型时，也能深刻感受到这个图形库的重要价值。

七、源代码组织结构

```
├── 外部依赖项/                  # 项目依赖的第三方库（SFML3.0.2）
├── 头文件/
|   ├── AppController.h        # 应用全局控制器（负责程序流程、模块调度）
|   ├── ControlPanel.h         # 控制面板模块（交互控件、操作逻辑声明）
|   ├── DataLogger.h           # 数据日志模块（数据采集、存储功能声明）
|   ├── EngineModel.h          # 引擎模型模块（引擎参数、运行逻辑声明）
|   ├── FaultDetector.h        # 故障检测模块（故障识别、诊断逻辑声明）
|   ├── FaultTypes.h           # 故障类型定义（枚举、常量、结构体声明）
|   ├── FuelSystem.h           # 燃油系统模块（燃油参数、控制逻辑声明）
|   ├── Gauge.h                # 仪表模块（数据可视化控件声明）
|   ├── IndicatorPanel.h       # 指示面板模块（状态显示控件声明）
|   └── TestScenarioGenerator.h  # 测试场景生成器（模拟测试数据逻辑声明）
├── 源文件/
|   ├── AppController.cpp     # AppController.h 的实现代码
|   ├── ControlPanel.cpp      # ControlPanel.h 的实现代码
|   ├── DataLogger.cpp        # DataLogger.h 的实现代码
|   ├── EngineModel.cpp       # EngineModel.h 的实现代码
|   ├── FaultDetector.cpp     # FaultDetector.h 的实现代码
|   ├── FuelSystem.cpp        # FuelSystem.h 的实现代码
|   ├── Gauge.cpp             # Gauge.h 的实现代码
|   ├── IndicatorPanel.cpp    # IndicatorPanel.h 的实现代码
|   └── TestScenarioGenerator.cpp  # TestScenarioGenerator.h 的实现代码
|   ├── main.cpp
```